LLD-10: Collections

LLD-10: Python Advanced-2 — Collections

> Python's built-in list and dict are great. But for specific problems, the `collections` module has purpose-built tools that are faster, cleaner, and less buggy.

Recap — Typing, Generics & AI Agents

> **Question 1:** `find_user(999)` returns `None`. You call `user["name"]` without checking. When does the bug surface?

Answer

At runtime in production. Python doesn't enforce types — it happily runs `user["name"]` and crashes with `TypeError`. mypy would catch it *if you run it*, but mypy is a separate tool, not part of Python's execution. That's why you need mypy in CI/CD.

> **Question 2:** `def first(items: list[T]) -> T` — you call `first(["apple", "banana"])`. What type does the IDE show?

Answer

`str` — TypeVar `T` is inferred as `str` from `list[str]`. The return type `T` becomes `str`. The IDE offers `.upper()`, `.split()` etc. The type flows through the TypeVar.

> **Question 3:** You want your AI coding agent (Claude, Cursor, Copilot) to always use type hints. Where do you put these instructions?

Answer

In a **`CLAUDE.md`** (for Claude Code) or **`.cursorrules`** (for Cursor) file in the project root. The agent reads it automatically at the start of every session — like a persistent system prompt for your project. Rules like "always use type hints", "use dataclasses not plain dicts" apply to ALL code the agent writes.

> **Question 4:** Your team has a pre-commit hook running mypy. A developer writes code without type hints. What happens at `git commit`?

Answer

The commit is REJECTED. mypy returns a non-zero exit code (errors found), git refuses to create the commit. The developer must fix the type errors first. Layer 1: pre-commit hooks. Layer 2: CI/CD on the PR. Type-broken code never reaches main.

Recap — Protocol (From Last Class)

Protocol is structural typing — "if it has the right methods, it's valid." No inheritance needed.

```python
from typing import Protocol

class Drawable(Protocol):
    def draw(self) -> str: ...

# These DON'T inherit from Drawable — they just have draw()
class Circle:
    def draw(self) -> str: return "○"

class Square:
    def draw(self) -> str: return "□"

# Accepts ANYTHING with draw() -> str
def render_all(shapes: list[Drawable]) -> None:
    for s in shapes:
        print(s.draw())

render_all([Circle(), Square()])  # Works! No inheritance needed.
```

	ABC	Protocol
Requires inheritance?	Yes	**No**
Works with 3rd-party?	No (must wrap)	**Yes**
Runtime check?	`isinstance()`	`@runtime_checkable`
Analogy	Java interface	**Go interface**
Use when	You own the hierarchy	You need a contract for any class

---

Why Not Just Use list and dict?

You can build a house with just a hammer and nails. But a screwdriver, level, and drill make you 10x faster and less error-prone.

How many times have you written these patterns?

- `if key not in d: d[key] = []` — before appending

- Counting items with a manual `+= 1` loop
- `list.insert(0, item)` — and wondering why it's slow
- `point[0], point[1]` — and forgetting which is x vs y
- Subclassing `dict` and finding `.update()` ignores your `__setitem__`

Every one has a **purpose-built solution** in Python's `collections` module:

Collection	Replaces	One-Line Description
defaultdict	`if key not in d: d[key] = []`	Dict with auto-created missing keys
Counter	Manual counting loops	Count things from any iterable
deque	`list.insert(0, x)` (O(n)!)	Fast append/pop from both ends
namedtuple	`point[0]` — what is 0?	Tuple with named fields
OrderedDict	Order-aware comparisons	Dict with order-sensitive equality
ChainMap	`{**defaults, **overrides}`	Layered dict lookups
UserDict/UserList	Broken `dict` subclasses	Safe base for custom collections

Plus **frozenset** (immutable set), **queue.Queue** (thread-safe), and more.

Python Collection Gotchas — Interview Favorites

> **Q:** What type is `a = {}`?

Answer

dict, not set! `{}` is always a dict. For an empty set use `set()`.

> **Q:** `t = (42)` — what type is `t`?

Answer

int, not tuple! Single-element tuple needs a comma: `(42,)`. The parentheses are just grouping.

> **Q:** Can you do `{[1, 2]: "value"}`?

Answer

No — TypeError. Lists are mutable → unhashable → can't be dict keys. Use tuples: `{(1, 2): "value"}`. Same rule for set elements.

> **Q:** `def add(name, students=[]): students.append(name); return students` — call `add("Alice")` then `add("Bob")`. What does the second call return?

Answer

`["Alice", "Bob"]` — the default `[]` is created ONCE when the function is defined. Every call shares the SAME list. Fix: use `students=None` and create a new list inside.

> **Q:** `t = ([1, 2], 3)` — can you do `t[0].append(99)`?

Answer

Yes! Tuple immutability is shallow. You can't reassign `t[0] = [4, 5]`, but you CAN mutate the list inside it. The tuple holds the same list object — the list's contents changed, not the tuple's reference.

---

defaultdict — No More KeyError

The Problem

```
# Grouping words by first letter with plain dict:
grouped = {}
for word in words:
    first = word[0]
    if first not in grouped:     # boilerplate every time!
        grouped[first] = []
    grouped[first].append(word)
```

The Fix

```
from collections import defaultdict

grouped = defaultdict(list)   # missing key → empty list
        automatically
for word in words:
    grouped[word[0]].append(word)   # no if-check!
```

Factory Functions

Factory	Missing key returns	Use case
`defaultdict(list)`	`[]`	Grouping items by key
`defaultdict(int)`	`0`	Counting occurrences
`defaultdict(set)`	`set()`	Collecting unique values
`defaultdict(lambda: "N/A")`	`"N/A"`	Custom default value

> **Gotcha:** Accessing a missing key **creates** it. `d["x"]` adds `"x"` to the dict even if you're just reading. Use `d.get("x")` or `"x" in d` to check without creating.

Counter — Count Everything

```python
from collections import Counter

words = "the cat sat on the mat the cat".split()
counts = Counter(words)
# Counter({'the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'mat': 1})

counts['the']        # 3
counts['dog']        # 0 (not KeyError!)
counts.most_common(2)  # [('the', 3), ('cat', 2)]
```

Real-World: Vote Counting

```python
votes = ["Alice", "Bob", "Alice", "Charlie", "Alice", "Bob"]
vote_counts = Counter(votes)
# Counter({'Alice': 3, 'Bob': 2, 'Charlie': 1})

winner = vote_counts.most_common(1)[0]
print(f"Winner: {winner[0]} with {winner[1]} votes")
# Winner: Alice with 3 votes
```

Counter Arithmetic

```python
a = Counter(apples=3, bananas=2)
b = Counter(apples=1, bananas=4)

a + b  # Counter(apples=4, bananas=6)  — combine
a - b  # Counter(apples=2)             — subtract (drops ≤0)
a & b  # Counter(apples=1, bananas=2)  — minimum
a | b  # Counter(apples=3, bananas=4)  — maximum
```

> **Real-world:** word frequency, inventory tracking, vote counting, log analysis, feature usage metrics.

---

deque — Fast From Both Ends

The Problem

```python
# list.insert(0, x) is O(n) — shifts every element!
# For 100,000 items: list takes ~2s, deque takes ~0.002s
# deque is 1000x faster for prepend operations
```

The Fix

```python
from collections import deque

dq = deque([1, 2, 3])
```

```
dq.append(4)          # right: O(1)
dq.appendleft(0)      # left:  O(1)
dq.pop()              # right: O(1)
dq.popleft()          # left:  O(1)
```

maxlen — Sliding Window

```
recent = deque(maxlen=3)
for item in ["a", "b", "c", "d", "e"]:
    recent.append(item)
# Only last 3 kept: deque(['c', 'd', 'e'])
```

Performance Comparison

Operation	list	deque
Append right	O(1)	O(1)
Append left	**O(n)**	O(1)
Pop right	O(1)	O(1)
Pop left	**O(n)**	O(1)
Random access `[i]`	O(1)	**O(n)**

> **Use deque for:** queues (FIFO), sliding windows, BFS, recent-N items. **Don't use for:** random access by index.

---

namedtuple — Readable Tuples

The Problem

```
point = (28.6139, 77.2090)
point[0]  # What is this? x? latitude? id? WHO KNOWS!
```

The Fix

```
from typing import NamedTuple

class Point(NamedTuple):
    x: float
    y: float

p = Point(28.6139, 77.2090)
p.x       # 28.6139 — readable!
p[0]      # still works as tuple
p.x = 10  # AttributeError — immutable!
```

Useful Methods

```
user = User("Alice", 25, "alice@example.com")
user._replace(age=26)   # new User with age changed (original
        unchanged)
user._asdict()           # {'name': 'Alice', 'age': 25, 'email':
        '...'}
name, age, email = user  # unpacking works
```

namedtuple vs dataclass

	namedtuple	dataclass
Mutable?	No (immutable)	Yes (by default)
Is a tuple?	Yes (unpackable, indexable)	No
Methods?	Only _replace, _asdict	Full custom methods
Use for	Small immutable records	Larger structures, need methods

---

OrderedDict — Order-Sensitive Equality

```
from collections import OrderedDict

# Regular dict: order doesn't matter for ==
{"a": 1, "b": 2} == {"b": 2, "a": 1}  # True

# OrderedDict: order IS part of equality
OrderedDict([("a", 1), ("b", 2)]) == OrderedDict([("b", 2), ("a",
        1)])  # False!
```

Key methods: `move_to_end(key)`, `popitem(last=False)`. Great for **LRU caches**.

ChainMap — Layered Config

```
from collections import ChainMap

defaults = {"theme": "dark", "font_size": 14}
env = {"font_size": 16, "debug": True}
user = {"theme": "light"}

config = ChainMap(user, env, defaults)
config["theme"]      # "light" (user wins)
config["font_size"]  # 16 (env wins)
config["debug"]      # True (env)
```

Lookup order: first dict → second → third. No data copied. Real-world: Django settings layering, CLI arg overrides, template variable scopes.

---

frozenset — Immutable Set

```
# Sets can't be dict keys:
{{"python", "backend"}: "team_a"}  # TypeError!

# frozensets can:
tags = frozenset({"python", "backend"})
{tags: "team_a"}  # Works! frozenset is hashable.

# All set operations work (return new frozensets):
tags_a = frozenset({"python", "backend"})
tags_b = frozenset({"python", "ml"})
tags_a | tags_b  # union
tags_a & tags_b  # intersection
```

Use for: dict keys when key is a set, cache keys, graph edges, immutable permissions.

Custom Collections — UserDict, UserList, UserString

Why not subclass dict directly?

```
class BrokenUpperDict(dict):
    def __setitem__(self, key, value):
        super().__setitem__(key.upper(), value)

d = BrokenUpperDict()
d["hello"] = 1           # Uses __setitem__ → "HELLO"
d.update({"world": 2})   # BYPASSES __setitem__! → "world"
        (lowercase!)
# BUG: update() doesn't go through __setitem__ in dict!
```

The Fix: UserDict

```
from collections import UserDict

class UpperDict(UserDict):
    def __setitem__(self, key, value):
        super().__setitem__(key.upper(), value)

d = UpperDict()
d["hello"] = 1
```

```
d.update({"world": 2})   # Goes through __setitem__!
# {'HELLO': 1, 'WORLD': 2} — correct!
```

UserList — validated list (e.g., only positive numbers):

```
class PositiveList(UserList):
    def append(self, item):
        if item <= 0:
            raise ValueError(f"Only positive numbers! Got {item}")
        super().append(item)
```

UserString — custom string behavior (e.g., case-insensitive comparison):

```
class CaselessString(UserString):
    def __eq__(self, other):
        return self.data.lower() == str(other).lower()
```

> **Rule:** NEVER subclass `dict`, `list`, or `str` directly. Internal C methods bypass your overrides. Use `UserDict`, `UserList`, `UserString` instead.

Thread Safety in Collections

What's Safe Under the GIL?

Safe (single bytecode)	NOT Safe (compound operation)
`list.append(x)`	`counter += 1` (LOAD + ADD + STORE)
`list.pop()`	`if key not in d: d[key] = val` (check-then-act)
`dict[k] = v`	`list[i] = list[i] + 1`
`deque.append(x)`	Any read-modify-write
`deque.popleft()`	
`set.add(x)`	

For Thread-Safe Queues: `queue.Queue`

```
import queue

q = queue.Queue()
q.put("item")          # thread-safe, blocks if full
item = q.get()         # thread-safe, blocks if empty
q.task_done()
```

> **Don't rely on the GIL for thread safety.** It's a CPython implementation detail that may be removed (PEP 703 — free-threaded Python). Use `threading.Lock` or `queue.Queue` for correctness.

Which Collection for Which Job?

I need to…	Use
Group items by key	`defaultdict(list)`
Count occurrences	`Counter`
Fast prepend / pop from front	`deque`
Sliding window / last N items	`deque(maxlen=N)`
BFS queue	`deque` (popleft)
Immutable record with named fields	`NamedTuple`
Layered config (defaults + overrides)	`ChainMap`
Use a set as a dict key	`frozenset`
Order-sensitive equality	`OrderedDict`
LRU cache	`OrderedDict`
Custom dict behavior	`UserDict` subclass
Validated list	`UserList` subclass
Thread-safe queue	`queue.Queue`

Mutability Pairs

Mutable	Immutable
`list`	`tuple`
`set`	`frozenset`
`dict`	`MappingProxyType`
`bytearray`	`bytes`

Code Files

File	What It Demonstrates
`01_protocol_recap.py`	Protocol vs ABC — structural typing
`02_defaultdict.py`	defaultdict(list/int/set), gotchas
`03_counter.py`	Counter, most_common, arithmetic
`04_deque.py`	deque benchmark, maxlen, rotate
`05_namedtuple.py`	NamedTuple, _replace, _asdict, vs dataclass
`06_chainmap.py`	ChainMap layered config, new_child
`07_frozenset.py`	frozenset as dict key, set operations
`08_ordereddict.py`	OrderedDict equality, LRU cache
`09_custom_collections.py`	UserDict, UserList, UserString — why dict subclassing breaks
`10_thread_safety.py`	Safe vs unsafe operations, queue.Queue

File	What It Demonstrates
`11_which_collection.py`	Decision guide — which collection for which job

Key Takeaways

1. **defaultdict eliminates KeyError boilerplate** — factory functions (list, int, set) auto-create missing keys
2. **Counter counts anything** — most_common(), arithmetic, missing keys return 0
3. **deque is 1000x faster than list for prepend** — use for queues, BFS, sliding windows
4. **namedtuple = tuple + readability** — immutable, unpackable, typed
5. **Never subclass dict/list/str directly** — use UserDict/UserList/UserString
6. **GIL makes single operations atomic but don't rely on it** — use queue.Queue for thread-safe communication

Next class: Python Advanced-3 — Lambda Functions and FP

Resources — Official Python Docs

- collections module — defaultdict, Counter, deque, namedtuple, OrderedDict, ChainMap, UserDict/UserList/UserString
- frozenset — immutable set type
- queue.Queue — thread-safe queue
- typing.NamedTuple — typed namedtuple syntax
- Data Structures Tutorial — official tutorial on lists, dicts, sets, tuples